EDIA · PROJECT ORIENTATION

The missing *why*

Every team writes down what it decided. Almost nobody writes down why. We are building software that reads the meeting, finds the reasoning, and shows exactly what the written record left out.

Written for someone new to the project No prior context assumed Status: all 5 test meetings scored

Start with a real meeting

This is a genuine product-design meeting from a public research dataset. The team is choosing how to power a TV remote control. On the left is what people actually said. On the right is the entire written record of that meeting, as filed.

WHAT WAS SAID

Industrial Designer
We actually had an option of batteries, solar power, and a dynamo.

Industrial Designer
So I sort of picked battery.

Industrial Designer
Solar power's not available with the rubber case anyways.

User Interface
The dynamo would be interesting.

Project Manager
What about Kryptonite?

Marketing
If you throw it, it's gonna store loads of energy, and you don't need to buy a battery. But we need to find cost.

WHAT WAS WRITTEN DOWN

"This meeting was mainly about conceptual design... they had some discussions on physical attributes and the use of energy. They agreed that the screen was not necessary, and TV could be used as the screen instead. In addition, Kryptonite became their preferred energy."

Kryptonite was chosen	KEPT

Battery, solar and a dynamo were also considered	LOST
Solar was ruled out because it doesn't work with the rubber case	LOST
The Industrial Designer actually preferred battery	LOST
Nobody ever found what Kryptonite costs	LOST

The record is not wrong. It is just missing everything that would let a newcomer understand the decision. Six months later, when someone asks "why aren't we using a battery?", the answer is not recoverable from the document — and the person who knew has moved teams.

That gap between what was reasoned and what was recorded is the entire subject of this project.

Why this is worth building

Organisations make the same decision repeatedly because the first one left no trace of its reasoning. A rejected option gets re-proposed. A risk that someone flagged and accepted gets rediscovered as a surprise. A new joiner cannot tell the difference between a decision that was carefully weighed and one that nobody thought about.

The reasoning usually *does* exist — it was said aloud, and increasingly it is recorded or transcribed. It simply never survives the compression into the written record. If that loss is predictable, software can catch it and hand it back.

How the system works

Four stages. Each one is a separate piece of software that can be tested on its own.

1 **Read the files**

Open whatever the team actually has — PDFs, Word documents, transcripts, spreadsheets, plain text — and turn them into clean text the rest of the system can work with.

2

Cut the transcript into passages

A 40-minute meeting is too long to process at once, so it is cut into overlapping passages. Cuts always land between speakers, never mid-sentence — otherwise an objection gets separated from the answer to it, and the reasoning is destroyed by the cut itself.

3

Pull the reasoning out of each passage

A language model running locally on the laptop reads each passage and fills in a fixed form: what was decided, what else was considered, why the winner won, what conditions were attached, who objected, who signed off. Every entry must come with the exact words from the transcript that support it.

4

Check each piece against the written record

For every piece of reasoning found, ask: does the document contain this? Whatever the document does not contain is a gap, and gaps are what we report.

> **The one design idea worth understanding**
>
> Language models make things up. That is the central risk in a project like this: a system that invents plausible-sounding reasoning is worse than no system at all, because it is confidently wrong in exactly the place a human would trust it.
>
> So the model is required to paste the **exact words** from the transcript alongside every claim it makes. We then check, mechanically, whether those words actually appear in the source. No human judgement, no asking the model to grade itself. If the words are not there, it made them up, and we count it.

Where the project stands

Five stages of work, done in order because each one depends on the last being trustworthy.

1 · Confirm the problem is real DONE

We took five real meetings and compared, by hand, what was discussed against what was written. All five had lost reasoning — and the losses clustered in the same places every time: rejected options,

objections, and unresolved risks. What survived was the outcome and little else.

2 · Check a machine can tell the difference

DONE

We gave an off-the-shelf model 22 pieces of reasoning and asked, for each, whether the document contained it. It got 21 right and never missed a real gap. Its single mistake was calling something missing when the document had said the same thing in different words — a wording problem, not a reasoning problem, and one we have designed around ever since.

3 · Read the files

DONE

The file-reading module, covering five formats, with real tests against real files rather than a description of tests.

4 · Pull the reasoning out automatically

DONE

Complete: all five hand-checked meetings extracted and scored, 51 passages in total. This is the stage that currently limits the whole system — see the numbers below.

5 · Build a proper test set

TOOLING READY

Five meetings prove the problem exists but cannot prove a method works. We are building 55 meeting-and-document pairs where the correct answer is known in advance. All the software is written and tested; the five real meetings are already folded in. The 50 invented pairs have not been generated yet, and the human checking pass has not started.

What the numbers say so far

MEETING	FOUND (WORD MATCH)	FOUND (MEANING MATCH)	MADE-UP QUOTES
Casing shape & display	5 of 11 (45%)	7 of 11 (64%)	10%
Energy source	4 of 8 (50%)	7 of 8 (88%)	20%
Menu & case material	1 of 9 (11%)	2 of 9 (22%)	12%
Controller concepts	4 of 14 (29%)	9 of 14 (64%)	21%
Chip tier	2 of 6 (33%)	3 of 6 (50%)	18%
All five	**16 of 48 (33%)**	**28 of 48 (58%)**	**16%**

All five meetings, 51 passages. "Reasoning found" is shown twice because it depends entirely on how you decide two phrasings mean the same thing — word-matching is strict, meaning-matching is not. "Made-up quotes" does not depend on that choice, which is exactly why it is the number we trust most.

In plain terms: the system finds between a third and a half of the reasoning a careful human found, depending on how strictly you judge a match, and about one supporting quote in six is not a real quote. That is not good enough to put in front of a user, and we are not presenting it as if it were. It is a genuine baseline from a small model running on a laptop with no internet connection.

> **Why one column is nearly double the other**
>
> Both columns count the same extractions. They disagree only on what counts as a match. The strict column compares words: if the system writes "double-curved casing" and the reference says "double-curved casing — would block the scroll wheel", the overlap falls just under the bar and it scores zero. The other column compares meaning, and credits it.
>
> Neither is the honest number on its own, so both are reported. The gap between them — 25 percentage points — is itself the finding: it says most of what looks like failure is really disagreement about wording, and it is the same weakness we found in the detection model back in stage 2. We would rather show that than quietly pick the flattering column.

> **The most useful thing we learned this week**
>
> The third meeting scored worst of the five — 1 of 9 by word-matching, 2 of 9 by meaning — so we looked at why rather than averaging it away. That meeting opens with the team drawing animals on a whiteboard as an icebreaker, and settles its real decisions — a screen menu, a plastic case — right at the end.
>
> The system did the exact opposite of what was wanted. It confidently reported "the team decided to draw their favourite animals" as a decision, and then, on the two passages that contained the meeting's actual decisions, it reported that no decision had been made.
>
> So the failure is not random noise. The model over-reports where someone states something declaratively, and under-reports where a decision emerges gradually through discussion — which is how most real decisions are made. That points at a specific fix (a second, targeted pass over each passage) rather than the vague one (use a bigger model).

What is left to do

In the order it has to happen, because each item depends on the one above it.

Machine time — queued, no decisions needed

- **Generate the 50 invented meeting pairs.** Roughly six hours of processing, saved as it goes so it can be stopped and resumed.

Human time — cannot be automated, and should not be

- **Check the invented pairs by hand.** Around 500 individual judgements. The software decides what to leave out *before* writing either text, so a person is verifying an answer key rather than building one from scratch — much faster and much more reliable.
- **Have a second person independently check ten of them.** If two people disagree about what counts as missing, the whole test set is unsound. This measures that directly, and it has to be done blind — the second person never sees the first person's answers.

Still to build

- **Run gap detection across the full test set.** This is where all the earlier stages finally meet: 55 pairs instead of 22 hand-picked examples.
- **Combine repeated decisions.** A decision discussed in three passages currently produces three separate entries. Merging them is deliberately switched off for now, so that it cannot quietly flatter the current numbers.
- **Raise the amount of reasoning found** from roughly a third towards something usable — the main open problem.
- **Test owner and approver detection** on realistic data. The public meetings we use have no formal sign-off at all, so those fields are currently untestable; the invented corporate pairs are the first place they can be evaluated honestly.

Two habits worth knowing about

Limitations are written down before anyone asks. The public dataset we test on has no filed decision documents, so we substitute its meeting summaries and say so everywhere the numbers appear. It records no formal approvals, so we score those separately instead of pretending. Five meetings from one domain cannot support a general claim, and we do not make one.

Measurements are designed so they can embarrass us. Made-up quotes are detected by checking the source text mechanically, not by asking the model how confident it feels. When our first measurement made the system look worse than it deserved — it was counting harmless tidying of "um" and "uh" as invention — we split the measure in two and reported both, rather than picking whichever number was kinder.

Words we use a lot

Discussion / document	The two things we compare: the meeting as it happened, and the written record filed afterwards.
Reasoning element	One piece of thinking behind a decision — an option considered, a reason, a condition, an objection. This is the unit we count.
Gap	A reasoning element present in the discussion but absent from the document. Finding these is the product.
Passage (or "chunk")	A slice of transcript small enough for the model to process at once, cut at a speaker boundary.
Hallucination	The model stating something that is not in the source. Measured here by checking whether its supporting quotes really appear in the transcript.
Completeness	The share of the reasoning a careful human found that the software also found.
Ground truth	The known-correct answer a system is graded against. For five meetings it was built by hand; for the 50 invented pairs it is decided in advance by construction.
QMSum	The public research dataset of real recorded meetings that our five test meetings come from.
Agreement	How often two independent people give the same answer. If it is low, the task is too vague to be measured at all.